Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' test (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

MICRO-ADR — PROYEC-21: Juego Web de Naves Espaciales

1. CONTEXTO
Se requiere un juego arcade estilo años 80 completamente en HTML5, CSS y JS puro, sin dependencias de servidor, frameworks ni bundlers. El juego debe ejecutarse abriendo directamente `index.html` en un navegador (protocolo `file:///` o HTTP estático). El requerimiento especifica múltiples archivos JS separados (`game.js`, `controls.js`, `obstacles.js`), pero la evaluación de entorno obliga a analizar la viabilidad de módulos ES6 nativos.

EVALUACIÓN DE ENTORNO CRÍTICA:
El proyecto es `frontend_static` puro (HTML5 + CSS + JS sin servidor Node/Python). Si se sirve desde `file:///`, los módulos ES6 (`type='module'`, `import/export`) están BLOQUEADOS por CORS del navegador. Para garantizar compatibilidad universal sin requerir un servidor local, se adopta la estrategia de **archivos JS planos con IIFE (Immediately Invoked Function Expressions)** y variables globales de namespace (`window.GameEngine`, `window.Controls`, etc.), cargados con `<script src='...'>` convencionales en orden de dependencia. Esto permite la separación de archivos solicitada en la arquitectura SIN romper el entorno `file:///`.

2. DECISIÓN
Arquitectura elegida: Monolito Frontend Estático con Namespace Pattern (IIFE)
- Motor de juego basado en `requestAnimationFrame` + Canvas API 2D.
- Un único `index.html` que orquesta la carga de scripts en orden correcto.
- Separación lógica en archivos JS planos (sin `type='module'`), comunicados mediante un objeto global `SPACE_GAME` como namespace único.
- Todo el renderizado gráfico mediante Canvas 2D con arte ASCII/pixel art generado programáticamente (sin imágenes externas).
- Sin dependencias externas: cero npm, cero CDN de frameworks, cero archivos binarios.

3. JUSTIFICACIÓN BASADA EN YAGNI + KISS
- **¿Por qué NO microservicios/backend?** El juego es 100% client-side. No hay persistencia de scores en servidor, no hay multijugador, no hay autenticación. Agregar un backend violaría YAGNI de forma flagrante.
- **¿Por qué NO React/Vue/Phaser.js?** Phaser.js pesa ~1MB y resuelve problemas de juegos complejos 3D/física avanzada. Para un arcade 2D con Canvas, es sobreingeniería. Phaser introduciría una curva de aprendizaje y dependencia externa innecesaria.
- **¿Por qué NO ES6 Modules nativos?** Porque el criterio de entorno LOCAL prohíbe `type='module'`. La compatibilidad con `file:///` es prioritaria para un entorno sin servidor.
- **¿Por qué NO Web Workers?** El juego loop de un arcade 2D simple no requiere procesamiento paralelo. Un solo hilo con `requestAnimationFrame` a 60fps es suficiente.
- **¿Por qué NO localStorage para scores?** YAGNI: el requerimiento no pide persistencia entre sesiones. Se implementa solo score en memoria durante la sesión.
- **Arte programático vs imágenes:** Se generan sprites mediante Canvas 2D drawing API (formas geométricas, gradientes, efectos de brillo tipo CRT) para lograr el estilo arcade 80s SIN archivos `.png/.webp`. Esto elimina la dependencia de assets binarios y cumple la restricción del skill.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR
- **Riesgo 1 (BAJO):** Sin Web Audio API avanzada, los efectos de sonido serán generados con `AudioContext` oscillators (beeps sintéticos estilo 8-bit). Aceptable para el estilo arcade 80s requerido.

- **Riesgo 2 (BAJO):** El namespace global `SPACE_GAME` podría colisionar con otras librerías si se integra en una página existente. Mitigado porque el juego ocupa su propia página `index.html`.
- **Riesgo 3 (MEDIO):** Sin bundler, el orden de carga de scripts en `index.html` es crítico. Documentado explícitamente en el código con comentarios de dependencia.
- **Riesgo 4 (BAJO):** Responsive en móvil es limitado (controles táctiles no solicitados en criterios de aceptación). Se implementa viewport meta tag y canvas escalable, pero sin touch events en MVP.

2. MÉTRICAS DE ITERACIÓN (QA)

- js/config.js | Estado: APROBADO | Iteraciones: 1
- index.html | Estado: APROBADO | Iteraciones: 1
- js/utils.js | Estado: APROBADO | Iteraciones: 1
- js/audio.js | Estado: APROBADO | Iteraciones: 1
- js/controls.js | Estado: APROBADO | Iteraciones: 1
- js/particles.js | Estado: APROBADO | Iteraciones: 1
- js/obstacles.js | Estado: APROBADO (post-integration-fix) | Iteraciones: 1
- js/hud.js | Estado: APROBADO (post-integration-fix) | Iteraciones: 1
- js/game.js | Estado: APROBADO (post-integration-fix) | Iteraciones: 1
- css/styles.css | Estado: APROBADO | Iteraciones: 1
- css/responsive.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 381,886
Tokens de salida (output): 76,055
Total tokens: 457,941
Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~57.6 horas-hombre ahorradas

"H squad de 4 personas × 7.2 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“2,529 input | !‘1,713 output | 1 llamadas | ~1.8h ahorradas
MICROPLANIFICACION-DISEÑO: !“63,020 input | !‘28,974 output | 12 llamadas | ~21.9h ahorradas
FRONTEND-DESARROLLO: !“152,583 input | !‘28,114 output | 14 llamadas | ~17.7h ahorradas
test: !“163,754 input | !‘17,254 output | 16 llamadas | ~16.2h ahorradas